

CURSO DE PYTHON

Estruturas de Dados

Apostila passo a passo: da memória RAM ao projeto final

Python 3 · VS Code · estruturas do zero

APOSTILA ELABORADA POR

Professor Celso Brunno Rocha Custódio de Campos

Como esta apostila funciona

Você não pula para o simulador pronto. Em cada aula faz uma estrutura simples, lê o que aquilo significa e só então volta e edita. O projeto final nasce assim: array, lista, pilha, fila, hash, árvore, grafo e heap.

Uso desta apostila

© 2026 Celso Brunno Rocha Custódio de Campos. Material de uso didático.

É vedada a reprodução, distribuição ou comercialização sem autorização do autor.

O aluno pode usar o conteúdo para estudar e montar o projeto do curso.

Sumário

Clique no título para ir à seção.

Antes de começar	4
O que você vai construir	4
Ferramentas	4
Calendário (igual ao site)	4

INSTALAÇÃO

fazer em casa (não entra na aula)	6
Python 3	6
VS Code	6
Conferir se está pronto	6

Se deu erro, faça isto	7
Python não encontrado	7
IndentationError	7
Arquivo na pasta errada	7
NoneType e ponteiro	7

Boas práticas de programação	8
Nomes que explicam	8
Indentação e uma ideia por vez	8
Funções curtas	8
Não se repita	8
Comente o porquê, não o óbvio	8
Não copie sem entender	8
Checklist rápido (todas as aulas)	9

AULA 1

Fundamentos de Memória, Big-O e Arrays	10
Como a RAM guarda um array	10
Big-O em uma frase	11
Live coding: array estático	12

AULA 2

Listas Encadeadas (Linked Lists)	15
Nós e ponteiros	15
Trade-off de Big-O	16
Live coding: lista simples	16

AULA 3

Pilhas (Stacks) e Filas (Queues)	19
---	----

Pilha — LIFO	19
Fila — FIFO	19
Live coding: pilha e fila sobre nós.....	20

AULA 4

Tabelas Hash (Dicionários/Mapas)	23
O problema (antes da técnica).....	23
Resposta em uma frase	23
O dict do Python já é isso.....	23
Peças da tabela hash	24
Inserir e buscar (fluxo mental).....	24
Colisão — o ponto que mais confunde	25
Exercício na mão (antes do código)	25
Live coding: tabela com chaining	26

AULA 5

Árvores Binárias e Árvores de Busca (BST)	28
Vocabulário	28
Live coding: inserção e travessias.....	28

AULA 6

Grafos e Algoritmos de Travessia	31
Matriz versus lista de adjacência	31
BFS com fila	31

AULA 7

Heaps e preparação para o projeto	34
Índices no array.....	34
Briefing do Projeto Final	36

AULA 8

Projeto Final (PBL Integrado).....	38
O desafio	38
O que a avaliação cobra.....	39
Roteiro da apresentação (5 a 8 minutos)	39
Checklist antes de apresentar.....	40

APÊNDICE

Código de referência	41
Gabarito — Aula 4 (hash).....	41

APÊNDICE

Glossário de funções.....	43
Índice rápido.....	43

Antes de começar

Esta apostila acompanha o curso Estruturas de Dados. Cada aula tem 2h30. Você lê a explicação, copia o código com o professor e faz a prática no fim.

Usamos o VS Code do primeiro ao último encontro. A instalação é em casa — não consome tempo de aula. O projeto do curso é um simulador de logística: cada estrutura vira uma peça, e a Aula 8 junta o sistema.

O que você vai construir

Não é um script copiado da internet: você constrói em camadas. Uma peça por vez, conferida no checkpoint do fim da aula.

- ▶ Aulas 1 e 2 — memória contígua (array) e memória ligada (lista encadeada).
- ▶ Aulas 3 a 5 — regras de acesso (pilha/fila), busca por chave (hash) e ordem (BST).
- ▶ Aulas 6 e 7 — relações (grafo + BFS) e prioridade (heap).
- ▶ Aula 8 — simulador de logística, README e apresentação.

Dica

Não tente decorar. Digite o código. Se der erro, leia a mensagem em vermelho — ela quase sempre aponta a linha.

Atenção

Neste curso a `list` e o `dict` nativos do Python existem, mas não são o ponto. Você implementa a estrutura do zero para entender o custo. Usar `lista.append` sem saber o que acontece na memória é nota baixa.

Ferramentas

Python 3 e VS Code. O passo a passo está no capítulo Instalação — fazer em casa. Se a tela falhar, use Se deu erro, faça isto.

Não é um curso de algoritmo do zero. Antes da Aula 1 você precisa conseguir: criar um arquivo `.py`, usar `print`, `if` e `for`, e escrever uma classe simples. Lista e dicionário nativos aparecem só como contraste com a estrutura real. Funções pontuais (`ord`, `deque`, etc.) estão no *Apêndice — Glossário de funções* (nomes em teal sublinhado no texto são links).

Calendário (igual ao site)

- ▶ Aula 1 — Fundamentos de Memória, Big-O e Arrays
- ▶ Aula 2 — Listas Encadeadas (Linked Lists)

- ▶ Aula 3 — Pilhas (Stacks) e Filas (Queues)
- ▶ Aula 4 — Tabelas Hash (Dicionários/Mapas)
- ▶ Aula 5 — Árvores Binárias e Árvores de Busca (BST)
- ▶ Aula 6 — Grafos e Algoritmos de Travessia
- ▶ Aula 7 — Estruturas avançadas (Heaps) e preparação para o projeto
- ▶ Aula 8 — Projeto Final (PBL Integrado)

INSTALAÇÃO

fazer em casa (não entra na aula)

Atenção

Este capítulo *não* é aula. Nas 8 aulas o professor considera Python e VS Code já instalados. Use estas páginas em casa, antes do curso ou se o computador for novo.

Python 3

Passo 1. Abra python.org e baixe o instalador do Python 3.

Passo 2. No Windows, marque *Add python.exe to PATH* antes de clicar em Install Now.

Passo 3. Termine a instalação. Depois abra o Prompt e digite `python --version`. Deve aparecer 3.x.

VS Code

Passo 1. Baixe em code.visualstudio.com e instale.

Passo 2. Abra o VS Code → ícone de extensões → instale a extensão *Python* (Microsoft).

Passo 3. File → Open Folder na pasta do projeto (a pasta que contém `exemplos/`). Abra um `.py` e rode no terminal: `python exemplos/01_array.py`.

Dica

O terminal precisa estar *na pasta do projeto*. Se aparecer `FileNotFoundError`, você rodou de outra pasta. No VS Code: Terminal → New Terminal (ele abre na pasta que você abriu).

Conferir se está pronto

Passo 1. `python --version` — precisa ser 3.x

Passo 2. `python exemplos/01_array.py` — precisa mostrar o estado da memória e, no fim, um erro de capacidade.

Se deu erro, faça isto

Leia a *última* linha da mensagem em vermelho. Ache o sintoma abaixo e faça o X.

Python não encontrado

Sintoma: `python is not recognized`, comando não encontrado ou o VS Code pede para escolher um interpretador e a lista vem vazia.

- ▶ O Python não está instalado, ou foi instalado *sem* marcar Add python.exe to PATH.
- ▶ Feche o VS Code, instale de novo pelo capítulo de Instalação e *reabra* o programa.
- ▶ No VS Code: Ctrl+Shift+P → *Python: Select Interpreter* → escolha o Python 3.
- ▶ Teste fora do editor: abra o Prompt e digite `python --version`.

Passo a passo da instalação: [Instalação](#) (pág. 6)

IndentationError

Sintoma: `IndentationError: expected an indented block`. O Python usa os espaços da esquerda para saber o que está dentro do `if`, `for` ou `def`.

- ▶ Tudo que pertence ao `if/for/def` deve ficar 4 espaços mais à direita.
- ▶ Não misture Tab com espaço. No VS Code use só espaço (já vem assim).
- ▶ Se copiou código do WhatsApp, cole no editor e realinhe as linhas.
- ▶ A linha logo abaixo de `:` sempre precisa estar mais para dentro.

Arquivo na pasta errada

Sintoma: `FileNotFoundError` ao rodar `python exemplos/01_array.py`.

- ▶ O terminal tem que estar na pasta do projeto (aquela que contém `exemplos/`).
- ▶ No VS Code: File → Open Folder nessa pasta, depois Terminal → New Terminal.
- ▶ Não rode o arquivo de dentro da pasta `Downloads` se o projeto está em outro lugar.

NoneType e ponteiro

Sintoma: `'NoneType' object has no attribute 'proximo'` (ou `dado`, `anterior`). Você andou para além do último nó.

- ▶ Antes de usar `atual.proximo`, teste `if atual is not None`.
- ▶ Lista vazia: a cabeça (`cabeca`) é `None`. Não chame método nela.
- ▶ Esse erro é o equivalente ao `NullPointerException` de outras linguagens.

Boas práticas de programação

Código não é só para o computador. Daqui a uma semana você vai reler o que escreveu hoje. O professor também. Por isso esta apostila cobra hábito, não só o programa que “funciona”.

Nomes que explicam

Use snake_case: `inserir_no_inicio`, `faixa_atual`, `tabela_hash`. Função começa com verbo. Evite `x`, `temp`, `coisa`, `l2`.

Nomes

```
# ruim
n = n.p

# bom
atual = atual.proximo
```

Indentação e uma ideia por vez

Python usa 4 espaços. Uma linha, uma ideia. Não esprema inserção, ponteiro e print na mesma linha.

Funções curtas

Cada operação da estrutura faz uma coisa: inserir, remover, buscar, exibir. O `main` só demonstra. Isso vale da Aula 1 até o projeto.

Não se repita

Copiar o mesmo `while atual is not None` em quatro métodos é armadilha. Quando a lista muda, você corrige um e esquece os outros.

Comente o porquê, não o óbvio

Comentários

```
# ruim
self.topo = novo # atribui topo

# bom
# o novo nó passa a ser o topo: LIFO só mexe nesta ponta
self.topo = novo
```

Não copie sem entender

Atenção

Colar uma BST da internet sem saber apontar a regra esquerda/direita é nota baixa. Se não consegue explicar em voz alta por que escolheu hash em vez de árvore, ainda não é seu código.

Dica

No projeto, o README pede exatamente isso: onde está cada estrutura. Quem escreveu com clareza preenche o README em dois minutos.

Checklist rápido (todas as aulas)

- ▶ Nomes em português ou inglês, mas consistentes.
- ▶ Arquivo `.py` salvo; terminal na pasta do projeto.
- ▶ Erro em vermelho: leia a última linha da mensagem primeiro.
- ▶ Antes de apresentar: rode de novo do zero. O que não roda, não entrega.

AULA 1

Fundamentos de Memória, Big-O e Arrays

Objetivo desta aula (2h30)

Entender alocação contígua na RAM, ler Big-O no pior caso e simular um array estático em Python — inclusive o erro de capacidade cheia.

Para seguir, use o VS Code

A instalação não entra no tempo de aula. Se ainda não tiver o programa, faça o passo a passo em

Instalação (pág. 6)

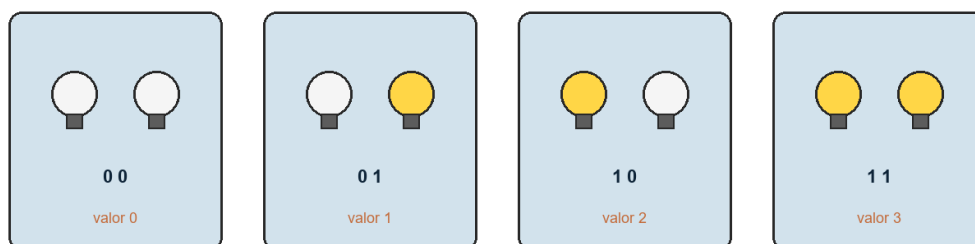
Se a tela falhar (Python não encontrado, pasta errada, NoneType), vá para **Se deu erro, faça isto** (pág. 7)

Como a RAM guarda um array

A memória RAM é um armário com gavetas numeradas (endereços). Um array estático pede um bloco de gavetas vizinhas. Se você declara capacidade 5, o sistema reserva exatamente 5 espaços contíguos.

Antes do array, o aluno precisa ver o tijolo: o bit. 2 bits são 4 combinações (lâmpada ligada/desligada). 32 bits de um `int` clássico têm teto; passar disso é overflow — em linguagens de tamanho fixo. O Python mascara isso; a estrutura de dados, não.

2 bits = 4 combinações = valores 0 a 3



Dois bits: quatro combinações. Cada lâmpada acesa vale um 1.

Quantos valores cabem em N bits?

Bits	Quantidade (2^N)	Sem sinal	Com sinal (aprox.)
2	4	0 a 3	-2 a 1
4	16	0 a 15	-8 a 7
8	256	0 a 255	-128 a 127
16	65 536	0 a 65 535	-32 768 a 32 767
32	4 294 967 296	0 a 4 294 967 295	-2 147 483 648 a 2 147 483 647

Faixas com e sem sinal. Na lousa, complete 8 e 16 bits com a turma antes de mostrar a tabela.

Array precisa de gavetas vizinhas (memoria continua)

lista = [1, 2, 3, 4, 5] vizinho ocupado impede crescer no lugar



Para caber o 6, o computador copia tudo para um bloco novo. Custo $O(n)$.

Cinco slots vizinhos. O bloco rosa já está ocupado: não dá para “empurrar” o 6 ali.

Se precisar do 6º item, a próxima gaveta pode estar ocupada por outro programa. Aí o computador procura um bloco de 6, copia os 5 itens e adiciona o novo. Essa cópia é o custo: tempo linear, $O(n)$.

append quando nao ha vizinho livre: copiar o bloco



O append que parece $O(1)$ às vezes copia a lista inteira. Por isso o “5 segundos / 6 segundos” da lousa.

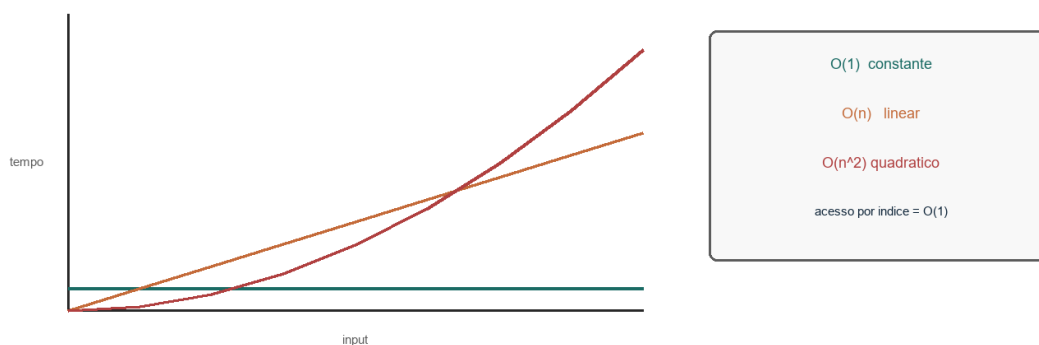
Atenção

A `list` do Python (`[]`) é um array dinâmico escrito em C. Ela mascara o redimensionamento. Nesta aula você simula o array rígido para ver o limite.

Big-O em uma frase

Big-O descreve o pior caso quando a entrada cresce. Não é “segundos no seu PC”: é como o trabalho cresce.

- ▶ $O(1)$ — constante. Acesso por índice: `endereço_base + (índice * tamanho_do_dado)`.
- ▶ $O(n)$ — linear. Inserir no meio exige empurrar os elementos seguintes.
- ▶ $O(n^2)$ — quadrático. Dois laços aninhados sobre a mesma coleção (aparece como contraste).

Big-O: como o tempo cresce quando o input cresce

O eixo horizontal é o tamanho da entrada; o vertical é o trabalho. $O(1)$ é a reta baixa; $O(n^2)$ dispara.

Live coding: array estático

Crie `exemplos/01_array.py`. Force o `OverflowError`: os alunos precisam ver o erro para entender a rigidez da estrutura.

`exemplos/01_array.py`

```
class ArrayEstatico:
    def __init__(self, capacidade):
        self.capacidade = capacidade
        self.tamanho_atual = 0
        self.dados = [None] * capacidade

    def acessar(self, indice):
        if 0 <= indice < self.tamanho_atual:
            return self.dados[indice]
        raise IndexError("Índice fora dos limites.")

    def inserir_no_final(self, valor):
        if self.tamanho_atual < self.capacidade:
            self.dados[self.tamanho_atual] = valor
            self.tamanho_atual += 1
        else:
            raise OverflowError("Array cheio.")

    def exibir(self):
        print("Estado da memória:", self.dados)

lista = ArrayEstatico(5)
lista.inserir_no_final(10)
lista.inserir_no_final(20)
lista.inserir_no_final(30)
lista.exibir()
print("posição 1:", lista.acessar(1))
```

Dica

Se `acessar` receber um índice dentro da capacidade mas além de `tamanho_atual`, o slot ainda é `None`. Distinguir “existe na memória” de “já foi preenchido” é o ponto da aula.

Boa prática

Sempre teste o limite: um array que nunca enche não ensina capacidade.

Práticas da aula

Prática A — Sistema de inventário restrito

Vocês foram contratados para o inventário de um jogo clássico. O personagem tem uma mochila com exatamente 4 slots. Dá para adicionar, listar e recusar item extra sem travar o jogo.

exemplos/01_mochila.py

```
class MochilaRPG:
    def __init__(self, slots):
        self.slots = slots
        self.itens_guardados = 0
        self.inventario = [None] * slots

    def coletar_item(self, item):
        print(f"Tentando coletar: {item}...")
        if self.itens_guardados < self.slots:
            self.inventario[self.itens_guardados] = item
            self.itens_guardados += 1
            print(f"[Sucesso] {item} no slot {self.itens_guardados - 1}.")
        else:
            print(f"[Falha] Mochila cheia! Descarte algo para pegar {item}.")

    def abrir_mochila(self):
        print("--- Conteúdo da Mochila ---")
        for i in range(self.slots):
            status = self.inventario[i] or "[Vazio]"
            print(f"Slot {i}: {status}")

mochila = MochilaRPG(4)
for item in ("Poção de Cura", "Espada de Ferro", "Mapa Antigo", "Tocha"):
    mochila.coletar_item(item)
mochila.abrir_mochila()
mochila.coletar_item("Amuleto Mágico")
```

Prática B — Inserir no meio (discussão)

No caderno (não precisa implementar agora): se a mochila tivesse que abrir o slot 0 para um item novo, quantos elementos andam? Isso é $O(n)$. Guarde essa resposta — é o gancho da Aula 2.

Antes da próxima aula, você precisa conseguir

- ▶ Explicar por que o acesso por índice é $O(1)$ e a inserção no meio é $O(n)$.
- ▶ Rodar o array estático e provocar o estouro de capacidade.
- ▶ Dizer o que aconteceria se a mochila ganhasse um “upgrade” de slots usando só array estático.

AULA 2

Listas Encadeadas (Linked Lists)

Objetivo desta aula (2h30)

Quebrar a contiguidade da RAM: montar nós com ponteiros, inserir no início em $O(1)$ e navegar para frente e para trás numa lista dupla.

Para seguir, use o VS Code

A instalação não entra no tempo de aula. Se ainda não tiver o programa, faça o passo a passo em

Instalação (pág. 6)

Se a tela falhar (Python não encontrado, pasta errada, NoneType), vá para **Se deu erro, faça isto** (pág. 7)

Retome a pergunta da Aula 1: como aumentar a mochila sem um bloco contínuo novo?

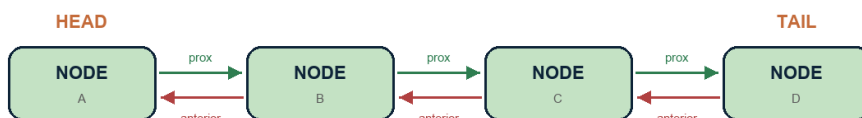
Nós e ponteiros

A lista encadeada espalha os dados em qualquer gaveta livre. Para não se perder, cada nó guarda duas coisas: o dado e o endereço do próximo nó.

- ▶ Simplesmente encadeada — mão única. Cada nó só conhece o próximo.
- ▶ Duplamente encadeada — mão dupla. Conhece o anterior e o próximo (gasta mais memória).

Lista simplesmente encadeada (prox so para a frente)

HEAD aponta o primeiro nó; TAIL o último; o prox do último é None.

Lista dupla: prox (verde) e anterior (vermelho)

Seta verde = próximo. Seta vermelha = anterior. É o custo extra de memória da lista dupla.

Trade-off de Big-O

- ▶ Acesso ao k-ésimo item piora: vira $O(n)$ — é preciso andar nó a nó.
- ▶ Inserção/remoção no início (com o ponteiro na mão) vira $O(1)$: só troca referências.

Live coding: lista simples

exemplos/02_lista.py

```
class No:
    def __init__(self, dado):
        self.dado = dado
        self.proximo = None

class ListaEncadeada:
    def __init__(self):
        self.cabeca = None

    def inserir_no_inicio(self, dado):
        novo = No(dado)
        novo.proximo = self.cabeca
        self.cabeca = novo
        print(f"[{dado}] inserido no início.")

    def exibir(self):
        atual = self.cabeca
        partes = []
        while atual is not None:
            partes.append(str(atual.dado))
            atual = atual.proximo
        print(" -> ".join(partes) + " -> None")

lista = ListaEncadeada()
lista.inserir_no_inicio("Terceiro")
lista.inserir_no_inicio("Segundo")
lista.inserir_no_inicio("Primeiro")
lista.exibir()
```

Dica

Inserir no início inverte a ordem de chegada. “Terceiro, Segundo, Primeiro” no código aparece como Primeiro → Segundo → Terceiro. Mostre isso na lousa.

Práticas da aula

Prática — Reprodutor de mídia

Playlist sem limite prévio de faixas, com botões Próximo e Anterior. Use lista duplamente encadeada.

Cuidado com `None` nas pontas.

ReprodutorStreaming — faixa atual no meio da playlist



A faixa atual (centro) conhece a anterior e a próxima — botões do player.

exemplos/02_reprodutor.py

```

class Faixa:
    def __init__(self, musica):
        self.musica = musica
        self.proximo = None
        self.anterior = None

class ReprodutorStreaming:
    def __init__(self):
        self.faixa_atual = None

    def adicionar_na_fila(self, musica):
        nova = Faixa(musica)
        if self.faixa_atual is None:
            self.faixa_atual = nova
            print(f"Playlist iniciada: {musica}")
            return
        nova.anterior = self.faixa_atual
        nova.proximo = self.faixa_atual.proximo
        if self.faixa_atual.proximo:
            self.faixa_atual.proximo.anterior = nova
        self.faixa_atual.proximo = nova
        print(f"Faixa [{musica}] adicionada como próxima.")

    def proxima_faixa(self):
        if self.faixa_atual and self.faixa_atual.proximo:
            self.faixa_atual = self.faixa_atual.proximo
            print(f">> {self.faixa_atual.musica}")
        else:
            print(">> Fim da playlist.")

    def faixa_anterior(self):
        if self.faixa_atual and self.faixa_atual.anterior:
            self.faixa_atual = self.faixa_atual.anterior
            print(f"<< {self.faixa_atual.musica}")
        else:
            print("<< Início da playlist.")
  
```

```
player = ReprodutorStreaming()
player.adicionar_na_fila("Bohemian Rhapsody")
player.adicionar_na_fila("Stairway to Heaven")
player.adicionar_na_fila("Hotel California")
player.proxima_faixa()
player.proxima_faixa()
player.faixa_anterior()
```

Boa prática

Toda navegação testa `if self.faixa_atual and self.faixa_atual.proximo`. Sem isso, o `NoneType` aparece na hora da demo.

Antes da próxima aula, você precisa conseguir

- ▶ Desenhar três nós no papel e o ponteiro `cabeca`.
- ▶ Inserir no início de uma lista simples e exibir até `None`.
- ▶ Explicar por que o botão Anterior precisa do ponteiro `anterior`.

AULA 3

Pilhas (Stacks) e Filas (Queues)

Objetivo desta aula (2h30)

Restringir o acesso: LIFO na pilha e FIFO na fila. Na live coding as duas nascem sobre nós; na prática (Call Center) usamos `deque` e `list`, ambos em $O(1)$.

Para seguir, use o VS Code

A instalação não entra no tempo de aula. Se ainda não tiver o programa, faça o passo a passo em **Instalação** (pág. 6)

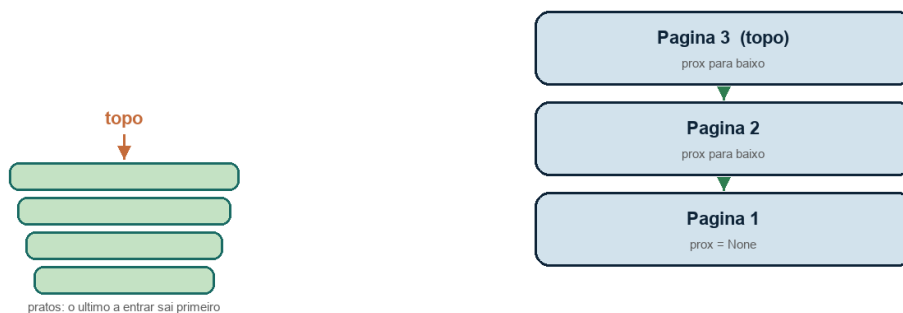
Se a tela falhar (Python não encontrado, pasta errada, NoneType), vá para **Se deu erro, faça isto** (pág. 7)

Nas Aulas 1 e 2 o tema era “onde a memória mora”. Agora o tema é a regra de negócio: de que ponta o dado entra e de que ponta sai.

Pilha — LIFO

Last In, First Out. Analogia: pilha de pratos. Só mexe no topo. Uso real: Ctrl+Z, botão Voltar do navegador, pilha de chamadas do Python.

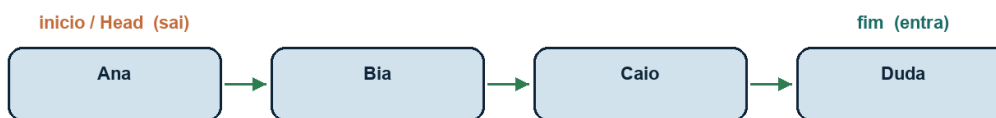
Pilha (LIFO) — so mexe no topo $O(1)$



Pratos e nós: só o topo muda. Empilhar e desempilhar são $O(1)$.

Fila — FIFO

First In, First Out. Analogia: fila de banco. Entra no fim, sai no início. Uso real: impressão, requisições de servidor, matchmaking de jogo.

Fila (FIFO) — entra no fim, sai no início $O(1)$ 

Matchmaking: quem chegou primeiro joga primeiro.

Ana entra primeiro e sai primeiro. No slide da aula isso aparece como Head + deque; aqui a fila nasce nos nós.

Atenção

Usar `lista.pop(0)` numa `list` do Python para simular fila é $O(n)$: todos os itens andam para a esquerda (Aula 1). Por isso a fila da live coding tem ponteiro de início e de fim na lista encadeada. Na prática do Call Center, `deque` resolve as duas pontas em $O(1)$; a `list` do histórico só empilha e desempilha no fim — também $O(1)$.

Live coding: pilha e fila sobre nós

exemplos/03_pilha_fila.py

```

class No:
    def __init__(self, dado):
        self.dado = dado
        self.proximo = None

class Pilha:
    def __init__(self):
        self.topo = None

    def empilhar(self, dado):
        novo = No(dado)
        novo.proximo = self.topo
        self.topo = novo

    def desempilhar(self):
        if self.topo is None:
            raise IndexError("Pilha vazia.")
        dado = self.topo.dado
        self.topo = self.topo.proximo
        return dado

    def vazia(self):
        return self.topo is None

class Fila:
    def __init__(self):
        self.inicio = None
        self.fim = None

```

```

def enfileirar(self, dado):
    novo = No(dado)
    if self.fim is None:
        self.inicio = self.fim = novo
    else:
        self.fim.proximo = novo
        self.fim = novo

def desenfileirar(self):
    if self.inicio is None:
        raise IndexError("Fila vazia.")
    dado = self.inicio.dado
    self.inicio = self.inicio.proximo
    if self.inicio is None:
        self.fim = None
    return dado

def vazia(self):
    return self.inicio is None

```

Práticas da aula

Prática – Call Center integrado

Vocês foram contratados para o balcão de um Call Center. Clientes entram numa fila de espera e são atendidos na ordem de chegada. O atendente às vezes encerra o chamado sem querer: precisa de uma pilha de histórico para desfazer o último encerramento e devolver o cliente ao início da fila (não ao fim).



appendleft: Maria volta para o INICIO da fila, não para o fim.

Fila = quem espera (FIFO). Pilha = histórico do atendente (LIFO). Desfazer usa appendleft.

exemplos/03_callcenter.py

```

from collections import deque

class CallCenter:
    def __init__(self):
        self.fila_espera = deque()    # Fila: quem chega
        self.historicoacoes = []     # Pilha: desfazer (Ctrl+Z)

```

```

def novo_cliente(self, nome):
    self.fila_espera.append(nome)
    print(f"[Entrada] Cliente {nome} entrou na fila de espera.")

def atender_proximo(self):
    if len(self.fila_espera) > 0:
        cliente_atendido = self.fila_espera.popleft() # O(1)
        self.historico_acoes.append(cliente_atendido)
        print(f"[Atendimento] Atendendo cliente: {cliente_atendido}")
    else:
        print("[Atendimento] A fila de espera está vazia.")

def desfazer_encerramento(self):
    if len(self.historico_acoes) > 0:
        cliente_recuperado = self.historico_acoes.pop() # O(1)
        self.fila_espera.appendleft(cliente_recuperado)
        print(f"[Desfazer] Erro! {cliente_recuperado} voltou ao início da fila.")
    else:
        print("[Desfazer] Não há ações recentes para desfazer.")

def mostrar_painel(self):
    print(f"\nPainel - Fila atual: {list(self.fila_espera)}")
    print("-" * 30)

sistema = CallCenter()
sistema.novo_cliente("João")
sistema.novo_cliente("Maria")
sistema.novo_cliente("Carlos")
sistema.mostrar_painel()
sistema.atender_proximo()
sistema.atender_proximo()
sistema.mostrar_painel()
sistema.desfazer_encerramento()
sistema.mostrar_painel()
sistema.atender_proximo()

```

Dica

Por que [appendleft\(\)](#) no desfazer, e não [append](#)? Se foi erro, a Maria não volta para o fim da fila atrás do Carlos: ela retoma a prioridade de quem já estava sendo atendido.

Antes da próxima aula, você precisa conseguir

- ▶ Dizer em uma frase a diferença LIFO / FIFO apontando a fila e a pilha do Call Center.
- ▶ Explicar por que [popleft\(\)](#) no [deque](#) é O(1) e [list.pop\(0\)](#) não é.
- ▶ Desfazer o encerramento da Maria e mostrar o painel com ela de volta na frente.

AULA 4

Tabelas Hash (Dicionários/Mapas)

Objetivo desta aula (2h30)

Entender o porquê do hash, mapear chave → índice, tratar colisão por encadeamento e buscar em tempo médio $O(1)$ — sem usar o `dict` nativo no catálogo da aula.

Para seguir, use o VS Code

A instalação não entra no tempo de aula. Se ainda não tiver o programa, faça o passo a passo em

Instalação (pág. 6)

Se a tela falhar (Python não encontrado, pasta errada, NoneType), vá para **Se deu erro, faça isto** (pág. 7)

Gancho da Aula 3: o gerente quer o cliente “Carlos” sem percorrer a fila inteira. Array é $O(1)$ por índice, mas o índice não é o CPF. Hash transforma a chave em índice.

O problema (antes da técnica)

Com array, achar `dados[3]` é instantâneo ($O(1)$): você já sabe o endereço. Na vida real a pergunta costuma ser outra: “qual o nome do usuário `u001`?” ou “qual o preço do produto `SKU-9981`?”. Aqui a chave *não* é o índice. Se você só tiver uma lista, precisa percorrer até achar → $O(n)$.

Pergunta da aula: como transformar uma chave qualquer (texto, id, CPF) em um índice rápido?

Resposta em uma frase

Hash = função que transforma a chave em um número; esse número vira o índice de um “balde” na tabela. Depois disso, você só procura *dentro daquele balde*, não na tabela inteira.

Analogia do prédio: imagine 5 andares (baldes 0 a 4). Chega a moradora com a chave “`u001`”. O porteiro (função de hash) calcula: “ela mora no andar 2”. Você sobe só o andar 2 e procura o nome na listinha daquele andar. Se duas pessoas caírem no mesmo andar, as duas ficam na listinha — isso é *colisão*. Ainda assim você não vasculha o prédio inteiro.

O dict do Python já é isso

```
peessoa = {"nome": "Bruna", "idade": "20"}  
print(peessoa["nome"]) # rápido — o dict já é hash
```

O `dict` nativo é uma tabela hash (bem otimizada). Nesta aula você *não* usa o `dict` para guardar o catálogo: recria a ideia na mão, para entender o que o Python faz por baixo.

- ▶ `dict` do Python — hash pronto, uso diário.

- ▶ **TabelaHash** da aula — hash didático, para estudar o mecanismo.

Peças da tabela hash

- ▶ **Tabela** — array de tamanho fixo (ex.: 5 ou 8 posições).
- ▶ **Balde** — cada posição é uma listinha (vazia no início).
- ▶ **Função de hash** — transforma a chave em inteiro.
- ▶ **Índice** — $\text{hash}(\text{chave}) \% \text{tamanho}$ (fica entre 0 e tamanho - 1).
- ▶ **Par** — dentro do balde guardamos `[chave, valor]`.

```
tamanho = 5

baldes:
[0] []
[1] []
[2] [ ["u001", "João"], ["outra", "..."] ] ← colisão
[3] [ ["u002", "Maria"] ]
[4] []
```

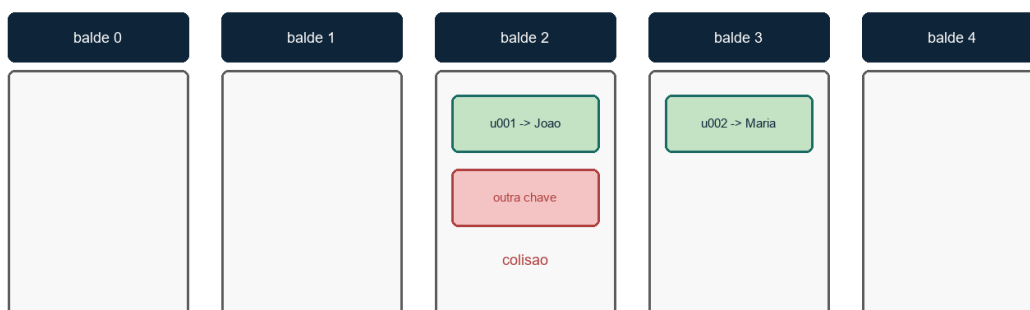
Fórmula didática do curso: $\text{indice} = (\text{soma dos códigos dos caracteres}) \% \text{tamanho}$.

```
def _hash(self, chave):
    # didático: soma dos códigos dos caracteres
    return sum(ord(c) for c in str(chave)) % self.tamanho
```

Dica

[`ord\(\)`](#) devolve o código numérico do caractere; [`sum\(\)`](#) soma esses códigos; [`%`](#) (resto) cabe o índice na tabela. Clique nos nomes para o glossário no final. Essa função de hash é só *para aprender* — a do `dict` real é mais sofisticada. O que importa agora é o fluxo: chave → número → balde → buscar/atualizar.

Tabela hash: $\text{indice} = \text{hash}(\text{chave}) \% \text{tamanho}$ + colisão por encadeamento



Cada balde é uma listinha. Duas chaves no mesmo índice = colisão; as duas ficam no encadeamento.

Inserir e buscar (fluxo mental)

Inserir "u001" → "João":

Passo 1. Calcular `indice = _hash("u001")`.

Passo 2. Ir ao `baldes[indice]`.

Passo 3. Se a chave *já existe* nesse balde → atualiza o valor (não duplica).

Passo 4. Se não existe → `append()` [`chave`, `valor`] na listinha.

Buscar "u001":

Passo 1. Calcular o *mesmo* índice.

Passo 2. Percorrer só aquele balde.

Passo 3. Se achar a chave → devolve o valor; senão → `None`.

Por que é rápido no caso médio? Porque a maioria dos baldes tem poucos itens — você quase não percorre a tabela toda.

Colisão — o ponto que mais confunde

Colisão = duas chaves *diferentes* geram o *mesmo* índice. Isso *não é erro*: é esperado. Tratamento desta aula: *encadeamento (chaining)* — cada balde é uma listinha; as chaves colidentes ficam juntas no mesmo balde (lista da Aula 2 em miniatura).

- ▶ Chaves bem espalhadas, tabela folgada → busca média $\approx O(1)$.
- ▶ Quase tudo no mesmo balde → piora para $O(n)$.

Atenção

Por isso o tamanho da tabela importa. Tabela pequena demais → mais colisões → mais lento. Tempo médio $O(1)$ *assume* tabela folgada e hash espalhada.

Comparação rápida (não misture com as outras aulas)

- ▶ Lista encadeada — inserir no início fácil; busca $O(n)$.
- ▶ Array por índice — $O(1)$ se souber o índice; índice $\neq$ CPF / id.
- ▶ Hash — busca por chave $\approx O(1)$ médio; não mantém ordem; há colisões.
- ▶ BST (Aula 5) — mantém ordem + busca $O(\log n)$; mais complexa.

No projeto final (logística): hash guarda `id_pacote` → `dados`; heap decide prioridade; grafo acha a rota. Hash acha a *ficha* do pacote — não decide quem sai primeiro (isso é heap).

Exercício na mão (antes do código)

Use tabela de tamanho 5 e a função do curso: `indice = sum(ord(c) for c in chave) % 5` (veja `ord()`, `sum()` e `%` no glossário). Códigos úteis: `u=117`, `0=48`, `1=49`, `2=50`, `a=97`, `b=98`.

Parte A — Calcular índices

Calcule o índice de "u001", "u002", "ab" e "ba". Anote: chave | soma dos ord | soma % 5 | balde.

Parte B — Desenhar a tabela

Insira "u001" → "João" e "u002" → "Maria". Desenhe os 5 baldes e o conteúdo de cada um.

Parte C — Forçar colisão

Ache duas chaves diferentes no mesmo balde. Sugestão: "ab" e "ba" têm os mesmos caracteres → mesma soma → colidem de propósito com esta função didática. Desenhe de novo o balde com dois pares.

Parte D — Busca mental

1) Para buscar "u001", quantos baldes você abre? 2) Se o balde tiver 2 pares, o que você compara? 3) Se o tamanho da tabela for 1, o que acontece com todas as inserções?

Dica

Gabarito no Apêndice — olhe só *depois* de tentar no papel.

Live coding: tabela com chaining

exemplos/04_hash.py

```

class TabelaHash:
    def __init__(self, tamanho=8):
        self.tamanho = tamanho
        self.baldes = [[] for _ in range(tamanho)]

    def _hash(self, chave):
        return sum(ord(c) for c in str(chave)) % self.tamanho

    def inserir(self, chave, valor):
        indice = self._hash(chave)
        for par in self.baldes[indice]:
            if par[0] == chave:
                par[1] = valor
                return
        self.baldes[indice].append([chave, valor])

    def buscar(self, chave):
        indice = self._hash(chave)
        for par in self.baldes[indice]:
            if par[0] == chave:
                return par[1]
        return None

```

```
catalogo = TabelaHash(5)
catalogo.inserir("u001", "João Silva")
catalogo.inserir("u002", "Maria Souza")
print("Busca u001:", catalogo.buscar("u001"))
print("Busca u999:", catalogo.buscar("u999"))
```

Boa prática

Ao inserir, primeiro percorra o balde: se a chave já existe, atualize o valor. Senão a mesma chave aparece duas vezes e a busca pega a antiga.

Práticas no computador

Prática — Catálogo de usuários / cache

Rode `python exemplos/04_hash.py`. Guarde `id` → nome (ou `url` → conteúdo). Busque por chave. Mude o tamanho para 5, imprima `catalogo.baldes` após cada inserção, force uma colisão e mostre os dois pares no mesmo balde. Explique em voz alta o que `_hash`, `inserir` e `buscar` fazem — sem olhar o código.

Antes da próxima aula, você precisa conseguir

- ▶ Por que hash existe, se já temos lista?
- ▶ O que significa `hash(chave) % tamanho`?
- ▶ O que é colisão e como o encadeamento resolve?
- ▶ Por que o tempo médio é $O(1)$, e quando isso falha?
- ▶ Em uma frase: diferença entre `dict` nativo e a `TabelaHash` da aula.

AULA 5

Árvores Binárias e Árvores de Busca (BST)

Objetivo desta aula (2h30)

Sair das estruturas lineares: inserir numa BST pela regra esquerda/direita e percorrer em ordem, pré-ordem e pós-ordem.

Para seguir, use o VS Code

A instalação não entra no tempo de aula. Se ainda não tiver o programa, faça o passo a passo em

Instalação (pág. 6)

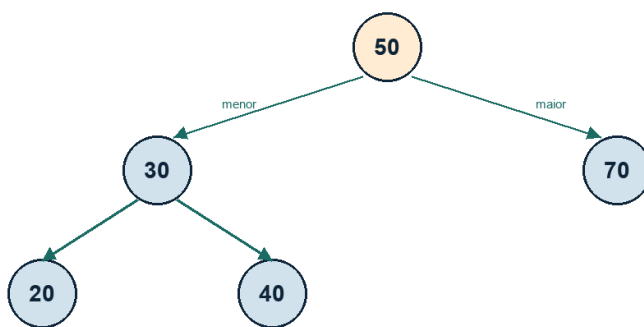
Se a tela falhar (Python não encontrado, pasta errada, NoneType), vá para **Se deu erro, faça isto** (pág. 7)

Hash acha pela chave, mas não entrega os dados ordenados. Árvore organiza hierarquia e, na BST, a busca média cai para $O(\log n)$ se estiver equilibrada.

Vocabulário

- ▶ Raiz — o primeiro nó. Folhas — nós sem filhos. Altura — níveis até a folha mais fundo.
- ▶ Árvore binária — cada nó tem no máximo dois filhos.
- ▶ BST — à esquerda, valores menores; à direita, maiores.

BST: esquerda < no < direita insercao 50, 30, 70, 20, 40



em-ordem visita esquerda, raiz, direita -> 20 30 40 50 70

Inserção 50, 30, 70, 20, 40. Em-ordem lê de baixo à esquerda até a direita: 20 30 40 50 70.

Atenção

Se você inserir 1, 2, 3, 4, 5 nessa ordem, a BST vira uma lista à direita. Aí a busca volta a ser $O(n)$. Equilíbrio (AVL/vermelho-preto) fica para depois; hoje o aluno precisa ver o caso ruim.

Live coding: inserção e travessias

exemplos/05_bst.py

```
class NoArvore:
    def __init__(self, valor):
        self.valor = valor
        self.esquerda = None
        self.direita = None

class ArvoreBusca:
    def __init__(self):
        self.raiz = None

    def inserir(self, valor):
        if self.raiz is None:
            self.raiz = NoArvore(valor)
        else:
            self._inserir(self.raiz, valor)

    def _inserir(self, no, valor):
        if valor < no.valor:
            if no.esquerda is None:
                no.esquerda = NoArvore(valor)
            else:
                self._inserir(no.esquerda, valor)
        elif valor > no.valor:
            if no.direita is None:
                no.direita = NoArvore(valor)
            else:
                self._inserir(no.direita, valor)

    def em_ordem(self, no):
        if no:
            self.em_ordem(no.esquerda)
            print(no.valor, end=" ")
            self.em_ordem(no.direita)

    def pre_ordem(self, no):
        if no:
            print(no.valor, end=" ")
            self.pre_ordem(no.esquerda)
            self.pre_ordem(no.direita)

    def pos_ordem(self, no):
        if no:
            self.pos_ordem(no.esquerda)
            self.pos_ordem(no.direita)
            print(no.valor, end=" ")

arvore = ArvoreBusca()
for codigo in (50, 30, 70, 20, 40):
    arvore.inserir(codigo)
print("Em ordem:")
arvore.em_ordem(arvore.raiz)
```

```
print("\nPré-ordem:")  
arvore.pre_ordem(arvore.raiz)
```

Dica

Em-ordem numa BST imprime crescente. Use isso como prova de que a regra esquerda/direita está certa.

Práticas da aula

Prática — Catálogo ordenado / categorias

Insira códigos de produto (ou nomes de categoria comparáveis) e liste em ordem. Opcional: pense no sistema de pastas do computador como analogia de hierarquia — a BST da prática é a versão com regra de busca.

Antes da próxima aula, você precisa conseguir

- ▶ Inserir cinco valores e desenhar a árvore no papel.
- ▶ Rodar em-ordem e conferir se saiu crescente.
- ▶ Explicar a diferença de objetivo entre hash ($O(1)$ médio) e BST (ordem + $O(\log n)$).

AULA 6

Grafos e Algoritmos de Travessia

Objetivo desta aula (2h30)

Modelar vértices e arestas, escolher lista de adjacência e achar o caminho mais curto em arestas com BFS — usando fila.

Para seguir, use o VS Code

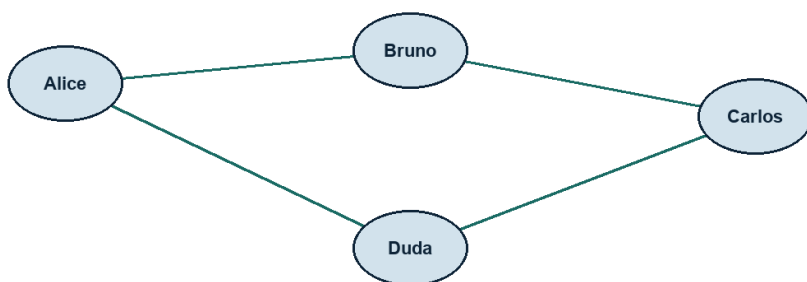
A instalação não entra no tempo de aula. Se ainda não tiver o programa, faça o passo a passo em

Instalação (pág. 6)

Se a tela falhar (Python não encontrado, pasta errada, NoneType), vá para **Se deu erro, faça isto** (pág. 7)

Árvore é um grafo sem ciclo, com um ancestral comum. Grafo permite ciclo: Alice é amiga de Bruno, Bruno de Carlos, Carlos de Alice.

Grafo: vértices (pessoas) e arestas (amizades)



BFS (fila): caminho mais curto em número de arestas. Alice → Carlos = Alice-Bruno-Carlos

Rede da prática: amizades são arestas. O caminho mais curto Alice–Carlos tem duas arestas.

Matriz versus lista de adjacência

- ▶ Matriz — tabela $V \times V$. Olhar se existe aresta é $O(1)$, mas gasta $O(V^2)$ de memória.
- ▶ Lista — para cada vértice, a lista de vizinhos. Melhor quando o grafo é esparsos (poucas arestas). É o que usamos no curso.

BFS com fila

Busca em largura visita camada por camada. A estrutura certa é a fila da Aula 3 (quem entra primeiro na visita sai primeiro). O caminho mais curto em número de arestas cai de graça se você guardar `veio_de`.

exemplos/06_grafo.py

```
from collections import deque
```

```

class Grafo:
    def __init__(self):
        self.vizinhos = {}

    def adicionar_vertice(self, nome):
        self.vizinhos.setdefault(nome, [])

    def adicionar_aresta(self, a, b):
        self.adicionar_vertice(a)
        self.adicionar_vertice(b)
        self.vizinhos[a].append(b)
        self.vizinhos[b].append(a)

    def bfs(self, inicio, destino):
        fila = deque([inicio])
        veio_de = {inicio: None}
        while fila:
            atual = fila.popleft()
            if atual == destino:
                break
            for vizinho in self.vizinhos[atual]:
                if vizinho not in veio_de:
                    veio_de[vizinho] = atual
                    fila.append(vizinho)
        if destino not in veio_de:
            return None
        caminho = []
        atual = destino
        while atual is not None:
            caminho.append(atual)
            atual = veio_de[atual]
        caminho.reverse()
        return caminho

rede = Grafo()
rede.adicionar_aresta("Alice", "Bruno")
rede.adicionar_aresta("Bruno", "Carlos")
rede.adicionar_aresta("Alice", "Duda")
print("Amigos de Bruno:", rede.vizinhos["Bruno"])
print("Caminho Alice → Carlos:", rede.bfs("Alice", "Carlos"))

```

Dica

Aqui o exemplo usa `deque` + `popleft()` para a BFS ficar $O(1)$ na ponta da fila. É a mesma regra FIFO que vocês implementaram com nós. No projeto, pode colar a classe `Fila` da Aula 3 no lugar do deque. Vértices novos usam `setdefault()`.

Práticas da aula

Prática — Rede social restrita

Cadastre 5 pessoas e algumas amizades. Mostre amigos em comum (interseção das listas) e o caminho mais curto entre dois usuários com BFS.

Antes da próxima aula, você precisa conseguir

- ▶ Desenhar o grafo da prática no papel (círculos e linhas).
- ▶ Rodar BFS e conferir o caminho com o desenho.
- ▶ Dizer por que DFS (pilha) não garante o menor número de arestas.

AULA 7

Heaps e preparação para o projeto

Objetivo desta aula (2h30)

Manter o menor (ou maior) elemento no topo com um heap em array, usar isso como fila de prioridade e receber o briefing do projeto final.

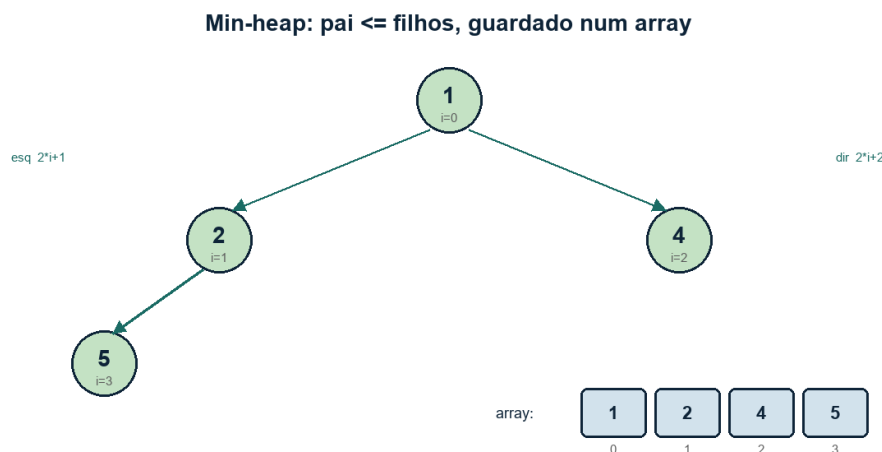
Para seguir, use o VS Code

A instalação não entra no tempo de aula. Se ainda não tiver o programa, faça o passo a passo em

Instalação (pág. 6)

Se a tela falhar (Python não encontrado, pasta errada, NoneType), vá para **Se deu erro, faça isto** (pág. 7)

Fila comum atende por ordem de chegada. Hospital, suporte e entrega urgente atendem por gravidade. Heap é a árvore (quase) completa guardada num array, em que $\text{pai} \leq \text{filhos}$ (min-heap).



A árvore e o array são a mesma estrutura. Filho esquerdo de i está em $2*i+1$.

Índices no array

- ▶ Filho esquerdo de i : $2*i + 1$. Filho direito: $2*i + 2$. Pai: $(i - 1) // 2$.
- ▶ Inserir: coloca no fim e “sobe” trocando com o pai enquanto for menor.
- ▶ Extrair o topo: pega o índice 0, põe o último no lugar e “desce”.

exemplos/07_heap.py

```
class MinHeap:
    def __init__(self):
        self.dados = []

    def _pai(self, i):
        return (i - 1) // 2
```

```

def _esq(self, i):
    return 2 * i + 1

def _dir(self, i):
    return 2 * i + 2

def _sobe(self, i):
    while i > 0:
        p = self._pai(i)
        if self.dados[i] < self.dados[p]:
            self.dados[i], self.dados[p] = self.dados[p], self.dados[i]
            i = p
        else:
            break

def _desce(self, i):
    n = len(self.dados)
    while True:
        menor = i
        e, d = self._esq(i), self._dir(i)
        if e < n and self.dados[e] < self.dados[menor]:
            menor = e
        if d < n and self.dados[d] < self.dados[menor]:
            menor = d
        if menor == i:
            break
        self.dados[i], self.dados[menor] = self.dados[menor], self.dados[i]
        i = menor

def inserir(self, valor):
    self.dados.append(valor)
    self._sobe(len(self.dados) - 1)

def extrair_min(self):
    if not self.dados:
        raise IndexError("Heap vazio.")
    minimo = self.dados[0]
    ultimo = self.dados.pop()
    if self.dados:
        self.dados[0] = ultimo
        self._desce(0)
    return minimo

heap = MinHeap()
for n in (5, 1, 4, 2):
    heap.inserir(n)
print("Extrai em ordem:", heap.extrair_min(), heap.extrair_min(), heap.extrair_min())

```

Dica

No Python do dia a dia existe `heapq` (min-heap nativo). Mostre um `heappush` / `heappop` no quadro

depois que o heap próprio funcionar. No projeto, vale a estrutura que você souber defender — a do zero prova que entendeu o índice.

Práticas da aula

Prática — Triage hospitalar

Pacientes entram com um número de gravidade. Quem tem o menor número (mais crítico) sai primeiro, mesmo tendo chegado depois. Use o heap. O contador `ordem` desempata quem tem a mesma gravidade (FIFO entre iguais).

exemplos/07_triagem.py

```
class Triage:
    def __init__(self):
        self.heap = MinHeap()
        self.ordem = 0

    def chegar(self, gravidade, nome):
        # tupla: menor gravidade numérica = mais urgente (1 = crítico)
        self.heap.inserir((gravidade, self.ordem, nome))
        self.ordem += 1
        print(f"{nome} entrou com gravidade {gravidade}.")

    def atender(self):
        gravidade, _, nome = self.heap.extrair_min()
        print(f"Atendendo {nome} (gravidade {gravidade})")

hospital = Triage()
hospital.chegar(5, "Paciente leve")
hospital.chegar(1, "Paciente crítico")
hospital.chegar(3, "Paciente moderado")
hospital.atender()
hospital.atender()
```

Briefing do Projeto Final

Na Aula 8 vocês recebem um simulador de logística. Peças obrigatórias:

- ▶ Grafo — mapa de bairros / pontos de entrega (BFS para o caminho).
- ▶ Tabela hash — dados do pacote pela chave `id`.
- ▶ Heap — ordem de saída pela urgência.
- ▶ Opcional: fila de veículos, pilha de “desfazer último despacho”, array de baús com capacidade.

Atenção

Quem chegar na Aula 8 sem hash, grafo e heap vai sofrer. Feche as práticas 4, 6 e 7. Não recomece do zero na apresentação.

Antes da próxima aula, você precisa conseguir

- ▶ Inserir quatro números no min-heap e extrair em ordem crescente.
- ▶ Atender o paciente crítico antes do leve.
- ▶ Escrever no caderno qual estrutura vai para cada peça do simulador.

AULA 8

Projeto Final (PBL Integrado)

Objetivo desta aula (2h30)

Juntar as estruturas num simulador de logística, defender as escolhas em 5 a 8 minutos e entregar um README que outra pessoa consiga rodar.

Para seguir, use o VS Code

A instalação não entra no tempo de aula. Se ainda não tiver o programa, faça o passo a passo em **Instalação** (pág. 6)

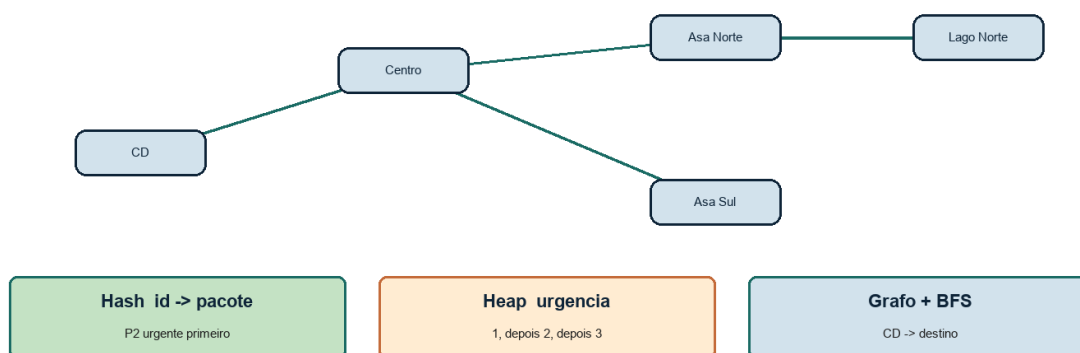
Se a tela falhar (Python não encontrado, pasta errada, NoneType), vá para **Se deu erro, faça isto** (pág. 7)

Atenção

Esta aula não é para inventar heap na hora. É para integrar, quebrar, consertar e apresentar.

O desafio

Há um centro de distribuição e bairros ligados por ruas (grafo). Pacotes têm id, destino e urgência. O sistema registra rotas, aceita pacotes e processa entregas do mais urgente para o menos urgente, mostrando o caminho no mapa.

Projeto final: tres pecas no mesmo sistema

Mapa (grafo) + ficha do pacote (hash) + ordem de saída (heap). P2 (urgência 1) sai primeiro.

exemplos/08_logistica.py

```
class SistemaLogistica:
    def __init__(self):
        self.rotas = Grafo()           # Aula 6
        self.pacotes = TabelaHash()    # Aula 4
        self.prioridade = MinHeap()    # Aula 7
        self.ordem = 0
```

```

def cadastrar_rota(self, origem, destino):
    self.rotas.adicionar_aresta(origem, destino)

def registrar_pacote(self, id_pacote, destino, urgencia):
    self.pacotes.inserir(id_pacote, {"destino": destino, "urgencia": urgencia})
    self.prioridade.inserir((urgencia, self.ordem, id_pacote))
    self.ordem += 1

def processar_entregas(self, deposito="CD"):
    while self.prioridade.dados:
        urgencia, _, id_pacote = self.prioridade.extrair_min()
        info = self.pacotes.buscar(id_pacote)
        caminho = self.rotas.bfs(deposito, info["destino"])
        print(f"{id_pacote} urgência {urgencia}: {caminho}")

# Monte um exemplo pequeno: 4 bairros, 3 pacotes, e rode processar_entregas.

```

O que a avaliação cobra

- ▶ Por que hash para o pacote e não uma BST (ou o contrário, se fizer sentido).
- ▶ Por que grafo no mapa — e não uma lista de endereços soltos.
- ▶ Por que heap na urgência — e não a fila FIFO da Aula 3.
- ▶ Um limite que você conhece: BST degenerada, colisão de hash, heap só no array.

Roteiro da apresentação (5 a 8 minutos)

- ▶ 1. Qual o problema? (pacotes com urgência + mapa da cidade).
- ▶ 2. Rodar o cadastro de rotas e de pacotes.
- ▶ 3. Processar entregas: o crítico sai primeiro; o caminho aparece.
- ▶ 4. No código: apontar hash, grafo/BFS e heap.
- ▶ 5. Uma frase do que você não fez de propósito (GPS real, banco, interface web).

README (entregue junto)

README.md

```

# Simulador de logística
O que é: entregas com urgência sobre um mapa em grafo.

Como o dado anda:
1. rotas (Grafo) — bairros e ruas
2. pacotes (TabelaHash) — id → destino e urgência
3. prioridade (MinHeap) — quem sai primeiro
4. BFS — caminho depósito → destino

```

```
Como rodar:  
python exemplos/08_logistica.py
```

Checklist antes de apresentar

Prática

Marque só o que o programa já faz de verdade (não o que você pretende fazer).

- ▶ ☐ Três ou mais pacotes; o de menor urgência numérica sai primeiro.
- ▶ ☐ BFS devolve um caminho que existe no grafo (teste com o desenho).
- ▶ ☐ Dá para apontar no código: hash, heap, grafo.
- ▶ ☐ README diz o que é, como rodar e o que não faz.
- ▶ ☐ Nomes de classe e arquivo fazem sentido para outra pessoa.

O que não precisa: banco de dados, site Django, API, login, mapa real de GPS.

APÊNDICE

Código de referência

Os arquivos abaixo estão na pasta `exemplos/`. Rode sempre com o terminal na pasta do projeto. Este apêndice é o mapa — o código completo está nos arquivos. Funções do Python (`ord`, `deque`, etc.) estão no *Apêndice — Glossário de funções*.

- ▶ `exemplos/01_array.py` e `01_mochila.py` — array estático e PBL da mochila.
- ▶ `exemplos/02_lista.py` e `02_reprodutor.py` — lista simples e lista dupla.
- ▶ `exemplos/03_pilha_fila.py` e `03_callcenter.py` — nós LIFO/FIFO e PBL do Call Center.
- ▶ `exemplos/04_hash.py` — tabela com encadeamento.
- ▶ `exemplos/05_bst.py` — inserção e travessias.
- ▶ `exemplos/06_grafo.py` — lista de adjacência e BFS.
- ▶ `exemplos/07_heap.py` e `07_triagem.py` — min-heap e prioridade.
- ▶ `exemplos/08_logistica.py` — simulador integrado (referência da Aula 8).

Dica

Se o seu projeto final divergir desta referência, tudo bem — desde que você aponte no código hash, grafo e heap e o programa rode do zero.

Gabarito — Aula 4 (hash)

Atenção

Só confira depois de tentar o exercício na mão da Aula 4.

Parte A — índices (tamanho 5):

```
"u001": ord('u')+ord('0')+ord('0')+ord('1') = 117+48+48+49 = 262
262 % 5 = 2 → balde 2

"u002": 117+48+48+50 = 263
263 % 5 = 3 → balde 3

"ab": 97+98 = 195
195 % 5 = 0 → balde 0

"ba": 98+97 = 195
195 % 5 = 0 → balde 0 (colide com "ab")
```

Parte B — depois de inserir `u001` e `u002`:

```
[0] []  
[1] []  
[2] [{"u001", "João"}]  
[3] [{"u002", "Maria"}]  
[4] []
```

Parte C — exemplo com *ab* / *ba* (colisão no balde 0):

```
[0] [{"ab", "..."}, {"ba", "..."}] ← colisão  
[1] []  
[2] [{"u001", "João"}]  
[3] [{"u002", "Maria"}]  
[4] []
```

Parte D — busca mental:

- ▶ Abre *um* balde (o do índice calculado).
- ▶ Compara a *chave* de cada par (`par[0]`), não o valor.
- ▶ Tabela de tamanho *1*: tudo cai no balde *0* → vira uma lista só → busca $O(n)$.

Professor: Celso Brunno Rocha Custódio de Campos. Curso de Python — Estruturas de Dados.

Glossário de funções

Ao longo da apostila, nomes em teal sublinhado (como [ord\(\)](#)) levam até aqui. Este glossário explica só o que o curso usa — não é manual completo do Python.

Dica

No Word/PDF, clique no nome da função no texto da aula para saltar à entrada. No sumário, abra este apêndice e use a lista abaixo.

Índice rápido

- ▶ [ord\(\)](#) 43
- ▶ [sum\(\)](#) 43
- ▶ [str\(\)](#) 44
- ▶ [len\(\)](#) 44
- ▶ [append\(\)](#) 44
- ▶ [pop\(\)](#) 44
- ▶ [deque](#) 44
- ▶ [appendleft\(\)](#) 44
- ▶ [popleft\(\)](#) 45
- ▶ [setdefault\(\)](#) 45
- ▶ [%](#) 45

ord()

Recebe *um* caractere e devolve o número inteiro que o representa na tabela Unicode/ASCII. No hash didático da Aula 4, somamos esses números para transformar texto em índice.

```
print(ord("A")) # 65
print(ord("u")) # 117
print(ord("0")) # 48
```

sum()

Soma os números de uma sequência. Em `sum(ord(c) for c in chave)`, o Python percorre cada caractere, chama [ord\(\)](#) e acumula o total.

```
print(sum([1, 2, 3])) # 6
print(sum(ord(c) for c in "ab")) # 97+98 = 195
```

str()

Converte o valor para texto (`str`). No hash usamos `str(chave)` para aceitar chave que não seja string (ex.: um número) sem quebrar o `for c in ....`

```
print(str(20))      # "20"
print(str("u001"))  # "u001"
```

len()

Devolve quantos itens há na coleção (lista, `deque`, string...). No heap e nas filas, `len(self.dados)` diz o tamanho atual.

```
print(len("abc"))    # 3
print(len([10, 20])) # 2
```

append()

Método de lista: coloca um item *no final*. Em hash, cada balde é uma listinha; `baldes[indice].append([chave, valor])` acrescenta o par naquele balde. Em pilha, `append` empilha no topo (fim da lista).

```
lista = [1, 2]
lista.append(3)
print(lista) # [1, 2, 3]
```

pop()

Remove e devolve um item. Sem argumento, remove o *último* ($O(1)$ na `list`) — é o desempilhar. `lista.pop(0)` remove o primeiro, mas na `list` do Python isso é $O(n)$ (todos andam). Por isso fila usa `deque` + `popleft()`.

```
pilha = [1, 2, 3]
print(pilha.pop()) # 3
print(pilha)       # [1, 2]
```

deque

Fila de duas pontas (`collections.deque`). Entra e sai nas duas extremidades em $O(1)$. No Call Center e no BFS usamos `deque` no lugar de `list`.

```
from collections import deque
fila = deque(["Ana", "Bia"])
fila.append("Caio")
print(fila.popleft()) # Ana
```

appendleft()

Método do `deque`: insere no *início* em $O(1)$. No Call Center, o desfazer usa `appendleft` para a Maria voltar à frente da fila (não ao fim).

```
from collections import deque
fila = deque(["Bia", "Caio"])
fila.appendleft("Maria")
print(fila) # deque(['Maria', 'Bia', 'Caio'])
```

`popleft()`

Método do `deque`: remove do *início* em $O(1)$. É o “próximo da fila” (FIFO) e o passo da BFS que tira o vértice da frente.

```
from collections import deque
fila = deque(["Ana", "Bia"])
print(fila.popleft()) # Ana
```

`setdefault()`

Método de `dict`: se a chave *não* existe, cria com o valor padrão e devolve; se já existe, só devolve o valor atual. No grafo: `self.vizinhos.setdefault(nome, [])` garante a listinha de vizinhos sem `if` extra.

```
g = {}
g.setdefault("CD", []).append("Centro")
print(g) # {"CD": ["Centro"]}
```

`%`

Operador *resto da divisão* (módulo). `262 % 5` vale 2: 5 cabe 52 vezes em 262 e sobram 2. No hash, `soma % tamanho` força o índice a ficar entre 0 e `tamanho - 1`.

```
print(10 % 3) # 1
print(262 % 5) # 2
print(7 % 7) # 0
```

Professor: Celso Brunno Rocha Custódio de Campos. Curso de Python — Estruturas de Dados.